一.必答问题

1.1 Maven 主要解决了什么问题?

- 依赖管理：在 `pom.xml` 中声明项目需要的依赖，Maven 会从仓库获取相应的 JAR 包，并处理这些依赖所需的其他依赖。
- 统一项目结构：约定源代码、配置文件和测试代码的存放位置，方便开发和维护。
- 标准化的跨平台自动化项目构建方式

1.2 HTTP 请求和响应分别由哪些主要部分组成?

HTTP请求协议

请求行

请求数据第一行（请求方式，资源路径，协议）

请求头

第二行开始，格式key:value

```
POST /brand HTTP/1.1
Accept: application/json, text/plain, */*
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Content-Length: 161
Content-Type: application/json;charset=UTF-8
Cookie: Idea-8296eb32=841b16f0-0cfe-495a-9cc9-d5aaa71501a6; JSESSIONID=0FDE4E430876BD9C5C955F061207386F
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/...

{"status":1,"brandName":"黑马","companyName":"黑马程序员","id":"","description":"黑马程序员"}
```

各个请求头的含义

Host	请求的主机名
User-Agent	浏览器版本，例如Chrome浏览器的标识类似Mozilla/5.0 ... Chrome/79，IE浏览器的标识类似Mozilla/5.0 (Windows NT ...) like Gecko
Accept	表示浏览器能接收的资源类型，如text/*，image/*或者*/*表示所有；
Accept-Language	表示浏览器偏好的语言，服务器可以据此返回不同语言的网页；
Accept-Encoding	表示浏览器可以支持的压缩类型，例如gzip，deflate等。
Content-Type	请求主体的数据类型。
Content-Length	请求主体的大小（单位：字节）。

请求体

POST请求，存放请求参数；请求头和请求体之间用一个空行隔开

- 对GET请求来说：请求参数在请求行中，没有请求体，GET请求大小在浏览器中是有限制的
- 对POST请求来说：请求参数在请求体中，POST请求大小是没有限制的

HTTP响应协议

响应数据分为三个部分：

响应行，响应头，响应体

- 响应行：响应数据的第一行（协议,状态码，描述）
- 响应头：第二行开始 格式key:value
- 响应体：最后一部分，存放响应数据

```
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 10 May 2022 07:51:07 GMT
Keep-Alive: timeout=60
Connection: keep-alive

[{id: 1, brandName: "阿里巴巴",companyName: "腾讯计算机系统有限公司", description: "玩玩玩"}]
```

常见响应状态码

1xx	响应中-临时状态码，表示请求已经接收，告诉客户端应该继续请求或者如果它已经完成则忽略它。
2xx	成功-表示请求已经被成功接收，处理已完成。
3xx	重定向-重定向到其他地方；让客户端再发起一次请求以完成整个处理。
4xx	客户端错误-处理发生错误，责任在客户端。如：请求了不存在的资源、客户端未被授权、禁止访问等。
5xx	服务器错误-处理发生错误，责任在服务端。如：程序抛出异常等。

状态码	英文描述	解释
200	OK	客户端请求成功，即**处理成功**，这是我们最想看到的状态码
302	Found	指示所请求的资源已移动到由Location响应头给定的 URL，浏览器会自动重新访问
304	Not Modified	告诉客户端，你请求的资源至上次取得后，服务端并未更改，你直接用你本地缓存
400	Bad Request	客户端请求有**语法错误**，不能被服务器所理解
403	Forbidden	服务器收到请求，但是**拒绝提供服务**，比如：没有权限访问相关资源
404	Not Found	**请求资源不存在**，一般是URL输入有误，或者网站资源被删除了
405	Method Not Allowed	请求方式有误，比如应该用GET请求方式的资源，用了POST
428	Precondition Required	**服务器要求有条件的请求**，告诉客户端要想访问该资源，必须携带特定的请求头
429	Too Many Requests	指示用户在给定时间内发送了**太多请求**（"限速"），配合 Retry-After(多长时间后可
431	Request Header Fields Too Large	**请求头太大**，服务器不愿意处理请求，因为它的头部字段太大。请求可以在减少请
500	Internal Server Error	**服务器发生不可预期的错误**。服务器出异常了，赶紧看日志去吧
503	Service Unavailable	**服务器尚未准备好处理请求**，服务器刚刚启动，还未初始化好

常见响应头的含义

Content-Type	表示该响应内容的类型，例如text/html，application/json。
Content-Length	表示该响应内容的长度（字节数）。
Content-Encoding	表示该响应压缩算法，例如gzip。
Cache-Control	指示客户端应如何缓存，例如max-age=300表示可以最多缓存300秒。
Set-Cookie	告诉浏览器为当前页面所在的域设置cookie。

1.3 IOC 和依赖注入的作用是什么?

控制反转：IOC

- 对象的创建控制权由程序自身转移到外部（容器）这里就是Spring容器
- 可以使用`@Component`注解将类交给IOC容器管理（是加在实现类上面而不是接口上面）

在本题中，Spring 扫描到带有 `@Service` 的 `EchoService` 和带有 `@RestController` 的 `EchoController` 后，会创建并管理相应的对象。

依赖注入：DI

- 容器为应用程序提供运行时所依赖的资源
- 具体是指当一个对象需要使用另一个对象时，由容器将它需要的对象传入。依赖注入是实现控制反转的一种方式。
- 使用`@Autowired`注解可以从IOC容器中找到该类型的bean，然后完成依赖注入

本题中`EchoController`需要调用`EchoService`，Spring 可以通过构造方法把`EchoService`对象传给它：

```
private final EchoService echoService;

@Autowired
public EchoController(EchoService echoService) {
    this.echoService = echoService;
}
```

他们可以：

- 集中管理组件对象及其依赖关系。
- 减少业务代码中手动创建对象的工作
- 减少层级之间的耦合度，方便后续的维护

1.4 Controller、Service 和持久层（Mapper/DAO）在完整项目中分别负责什么?

Controller:控制层，接受前端发送的请求，对请求进行处理，并响应数据

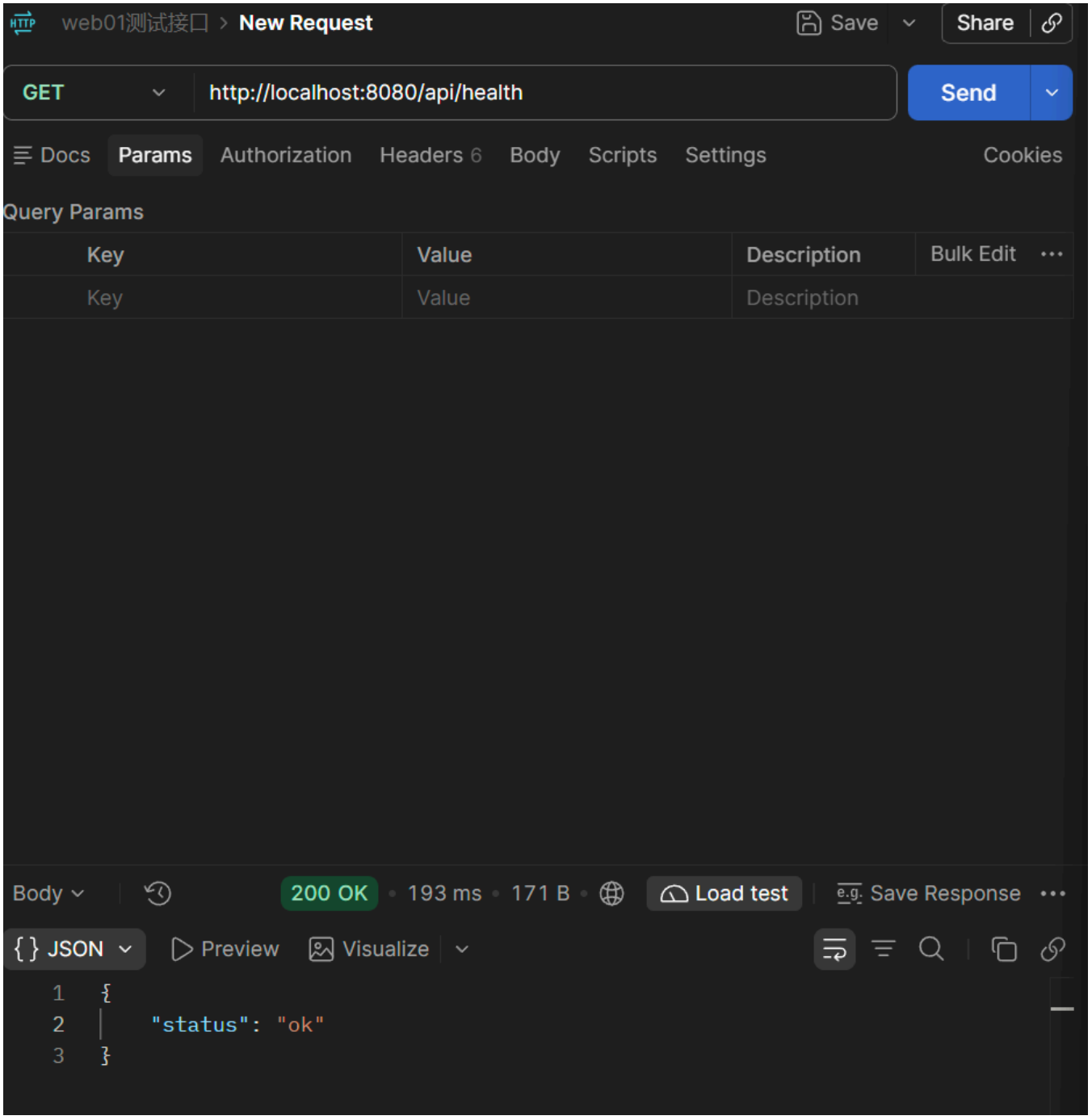
service:业务逻辑层，处理具体的业务逻辑

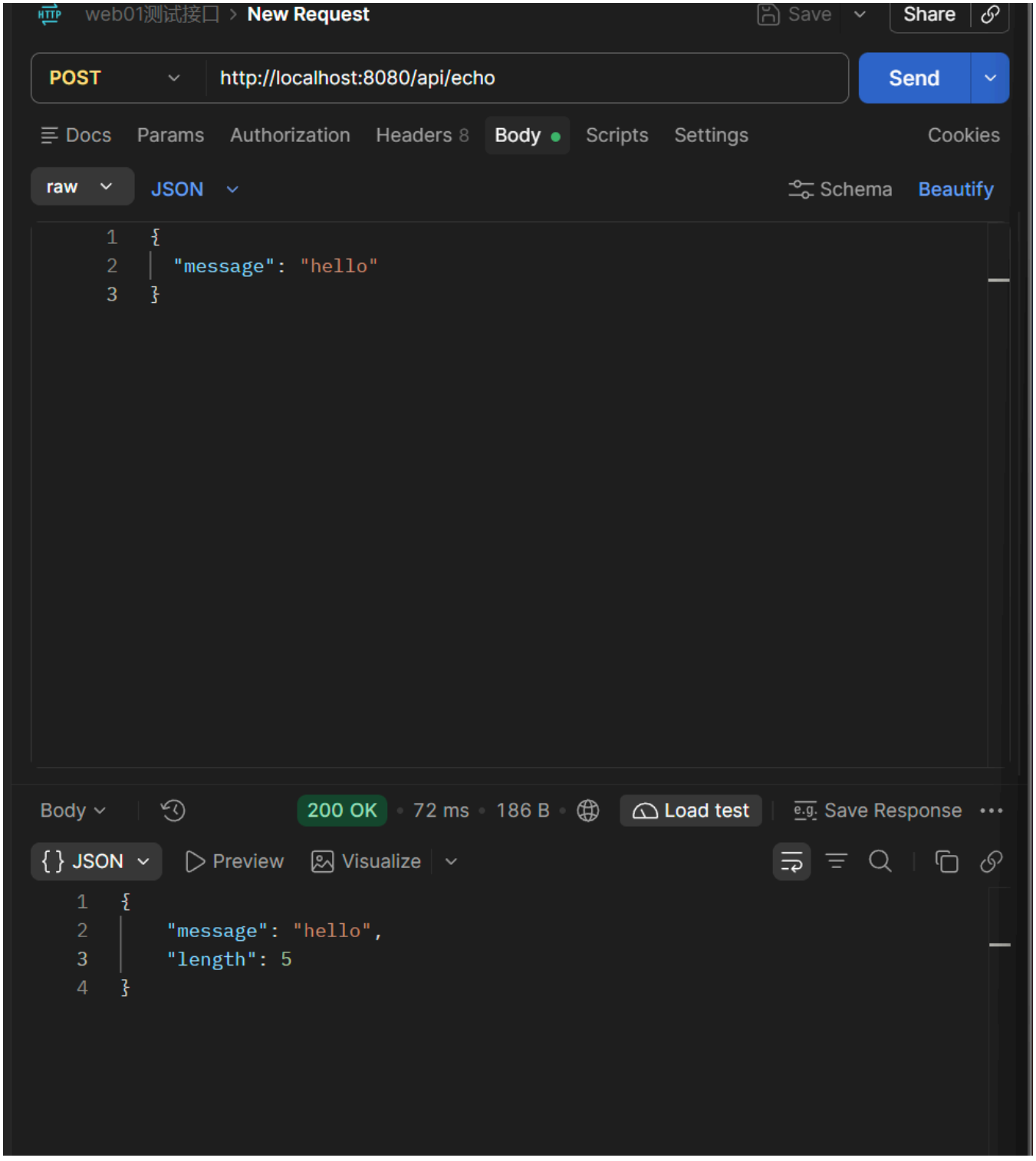
dao：数据访问层（Data Access Object）（持久层），负责数据访问操作，包括数据的增删改查

前端给请求->controller->调用Service->调用dao取得数据

dao的实现可以有文本，网络，数据库

1.5 测试接口结果


web01测试接口 > New Request
Save
Share
GET
http://localhost:8080/api/health
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
200 OK
193 ms
171 B
Load test
Save Response
JSON
Preview
Visualize
1 {
2 "status": "ok"
3 }



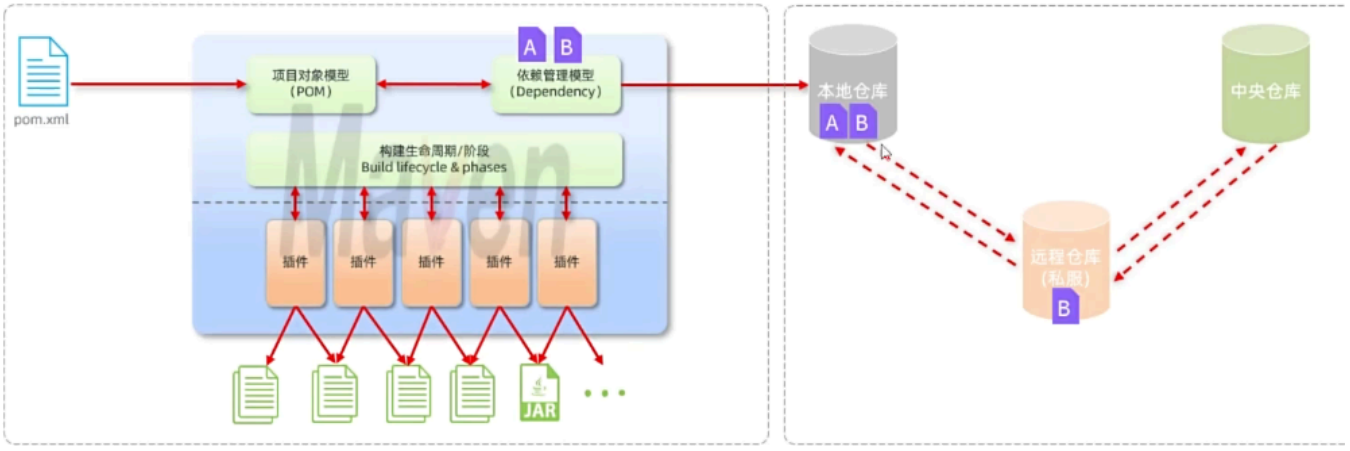
二.相关知识的笔记内容

1.1 Maven相关介绍

Maven是用来管理和构建Java项目的工具

- 方便快捷地管理项目依赖的资源(jar包)
- 标准化的跨平台自动化项目构建方式
- 提供标准，统一的项目结构

Maven结构



Maven的仓库用来存储和管理jar包

分为本地仓库，远程仓库和中央仓库

Maven坐标

- maven中的坐标是资源(jar)的唯一标识，通过该坐标可以唯一定位资源位置
- 使用坐标来定义项目或引入项目中需要的依赖

- Maven坐标的主要组成
  - groupId:定义当前Maven项目隶属组织的名称（域名反写）
  - artifactId:定义当前Maven项目名称（通常是模块名称 如order-service,goods-service)
  - version:当前版本号

1.2 Maven依赖管理

- 在pom.xml中编写标签
- 在标签中使用引入坐标
- 定义坐标的groupId,artifactId,version
- 刷新，引入最新加入的坐标

1.3 单元测试

2.1 SpringBoot入门

入门程序的剖析

起步依赖

- spring-boot-starter-webmvc-test：包含单元测试的需要的所有依赖
- spring-boot-starter-webmvc:包含web开发的所有依赖

运行main方法运行内部集成的tomcat服务器（依赖中包含的方法）

2.2 HTTP协议

- 超文本传输协议，浏览器与服务器之间数据传输的规则
- 基于TCP协议：面向连接，安全
- 基于请求-响应模型：一次请求对应一次响应
- HTTP协议是无状态的协议：对于事务处理没有记忆能力，每次的请求-响应是独立的
    - 缺点：多次请求之间无法共享数据
    - 优点：速度快
- URL:用来定位和访问资源的地址，平常说的“网址”就是一种 URL。
    - 比如：http://localhost:8080/hello?name=heima

部分	含义
`http://`	使用 HTTP 协议
`localhost`	访问本机
`8080`	访问本机的 8080 端口
`/hello`	URL 路径，指定要访问的接口
`?name=heima`	查询参数，传入 `name`，值为 `heima`

2.2.1 HTTP请求协议

请求行

请求数据第一行（请求方式，资源路径，协议）

请求头

第二行开始，格式key:value

```
POST /brand HTTP/1.1
Accept: application/json, text/plain, */*
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Content-Length: 161
Content-Type: application/json;charset=UTF-8
Cookie: Idea-8296eb32=841b16f0-0cfe-495a-9cc9-d5aaa71501a6; JSESSIONID=0FDE4E430876BD9C5C955F061207386F
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/...

{"status":1,"brandName":"黑马","companyName":"黑马程序员","id":"","description":"黑马程序员"}
```

各个请求头的含义

Host	请求的主机名
User-Agent	浏览器版本，例如Chrome浏览器的标识类似Mozilla/5.0 ... Chrome/79，IE浏览器的标识类似Mozilla/5.0 (Windows NT ...) like Gecko
Accept	表示浏览器能接收的资源类型，如text/*，image/*或者*/*表示所有；
Accept-Language	表示浏览器偏好的语言，服务器可以据此返回不同语言的网页；
Accept-Encoding	表示浏览器可以支持的压缩类型，例如gzip，deflate等。
Content-Type	请求主体的数据类型。
Content-Length	请求主体的大小（单位：字节）。

请求体

POST请求，存放请求参数；请求头和请求体之间用一个空行隔开

- 对GET请求来说：请求参数在请求行中，没有请求体，GET请求大小在浏览器中是有限制的
- 对POST请求来说：请求参数在请求体中，POST请求大小是没有限制的

2.2.2 HTTP协议请求数据获取

- HttpServletRequest对象里面封装了所有的请求信息

2.2.3 HTTP响应协议

响应数据分为三个部分：

响应行，响应头，响应体

- 响应行：响应数据的第一行（协议,状态码，描述）
- 响应头：第二行开始 格式key:value
- 响应体：最后一部分，存放响应数据

```
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 10 May 2022 07:51:07 GMT
Keep-Alive: timeout=60
Connection: keep-alive

[{id: 1, brandName: "阿里巴巴", companyName: "腾讯计算机系统有限公司", description: "玩玩玩"}]
```

常见响应状态码

1xx	响应中-临时状态码，表示请求已经接收，告诉客户端应该继续请求或者如果它已经完成则忽略它。
2xx	成功-表示请求已经被成功接收，处理已完成。
3xx	重定向-重定向到其他地方；让客户端再发起一次请求以完成整个处理。
4xx	客户端错误-处理发生错误，责任在客户端。如：请求了不存在的资源、客户端未被授权、禁止访问等。
5xx	服务器错误-处理发生错误，责任在服务端。如：程序抛出异常等。

状态码	英文描述	解释
200	OK	客户端请求成功，即**处理成功**，这是我们最想看到的状态码
302	Found	指示所请求的资源已移动到由Location响应头给定的 URL，浏览器会自动重新访问
304	Not Modified	告诉客户端，你请求的资源至上次取得后，服务端并未更改，你直接用你本地缓存
400	Bad Request	客户端请求有**语法错误**，不能被服务器所理解
403	Forbidden	服务器收到请求，但是**拒绝提供服务**，比如：没有权限访问相关资源
404	Not Found	**请求资源不存在**，一般是URL输入有误，或者网站资源被删除了
405	Method Not Allowed	请求方式有误，比如应该用GET请求方式的资源，用了POST
428	Precondition Required	**服务器要求有条件的请求**，告诉客户端要想访问该资源，必须携带特定的请求头
429	Too Many Requests	指示用户在给定时间内发送了**太多请求**（"限速"），配合 Retry-After(多长时间后可
431	Request Header Fields Too Large	**请求头太大**，服务器不愿意处理请求，因为它的头部字段太大。请求可以在减少请
500	Internal Server Error	**服务器发生不可预期的错误**。服务器出异常了，赶紧看日志去吧
503	Service Unavailable	**服务器尚未准备好处理请求**，服务器刚刚启动，还未初始化好

常见响应头的含义

Content-Type	表示该响应内容的类型，例如text/html，application/json。
Content-Length	表示该响应内容的长度（字节数）。
Content-Encoding	表示该响应压缩算法，例如gzip。
Cache-Control	指示客户端应如何缓存，例如max-age=300表示可以最多缓存300秒。
Set-Cookie	告诉浏览器为当前页面所在的域设置cookie。

2.2.4 HTTP响应数据设置

方法一

通过HttpServletResponse对象封装HTTP原始数据进行解析后的结果，可以直接调用API进行操作

通过调用该对象中的方法实现相应状态码（一般无需设置，浏览器处理后自动返回），响应头和响应体

方法二

通过ResponseEntity中的方法来完成配置

方式一：基于HttpServletResponse封装

```java
@RequestMapping("/response")
public void response(HttpServletResponse response) throws IOException {
    // 1.设置响应状态码
    response.setStatus(401);
    // 2.设置响应头
    response.setHeader("itheima","itheima");
    // 3.设置响应体
    response.getWriter().write("<h1>Hello Response</h1>");
}
```

方式二：基于ResponseEntity封装

```java
@RequestMapping("/response")
public ResponseEntity<String> response() {
    return ResponseEntity.status(401)          // 1.设置响应状态码
            .header("group", "itcast")         // 2.设置响应头
            .body("<h1>Hello Response</h1>"); // 3.设置响应体
}
```

2.3 分层解耦

2.3.1 三层架构

Controller:控制层，接受前端发送的请求，对请求进行处理，并响应数据

service:业务逻辑层，处理具体的业务逻辑

dao：数据访问层（Data Access Object）（持久层），负责数据访问操作，包括数据的增删改查

前端给请求->controller->调用Service->调用dao取得数据

dao的实现可以有文本，网络，数据库

2.3.2 IOC与DI入门

耦合：衡量软件中各个层/各个模块的依赖关联程度

内聚：软件中各个功能模块内部的功能联系

控制反转：IOC

- 对象的创建控制权由程序自身转移到外部（容器）这里就是Spring容器
- 可以使用`@Component`注解将类交给IOC容器管理（是加在实现类上面而不是接口上面）
    - `@Component`的衍生注解：

注解	说明	位置
@Component	声明bean的基础注解	不属于以下三类时，用此注解
@Controller	@Component的衍生注解	标注在控制层类上
@Service	@Component的衍生注解	标注在业务层类上
@Repository	@Component的衍生注解	标注在数据访问层类上（由于与mybatis整合，用的少）

依赖注入：DI

- 容器为应用程序提供运行时所依赖的资源
- 具体是指当一个对象需要使用另一个对象时，由容器将它需要的对象传入。依赖注入是实现控制反转的一种方式。
- 使用 `@Autowired` 注解可以从IOC容器中找到该类型的bean，然后完成依赖注入

Bean对象：IOC容器中创建，管理的对象